Master 1 CSIA / Matière : Vision par ordinateur

TP2

Installation :

numpy, matplotlib, skimage, **matplotlib.pyplot**

Téléchargement : télécharger l'image test .jfif

Définition :

Pour créer un vecteur numpy (un tableau qui contient les valeurs des pixels) de votre image vous pouvez utiliser **np.array()**.

Exemple :

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

img = Image.open( "/path/test.jfif" )

image_array = np.array( img )
print( 'classe :', type(image_array) )
print( 'type :', image_array.dtype )
print( 'taille :', image_array.shape )
print( 'modifiable :', image_array.flags.writeable )
```

Remarquez que nous avons généré un vecteur de classe **numpy.ndarray** contenant des valeurs de type **uint8** (entier) de 3 dimensions dont la forme est (hauteur, largeur, canal (Rouge, Vert, Bleu)). En plus, ce vecteur est modifiable (vous pouvez modifier les valeurs de chaque pixel). Si vous souhaitez créer un vecteur numpy non modifiable pour conserver par exemple votre image principale vous pouvez utiliser la commande **np.asarray()** :

Finalement, vous pouvez afficher l'image à partir du vecteur numpy en utilisant la fonction **plt.imshow()** :

```
plt.imshow( image_array )
plt.show()
```

Exercice 1 : Créer une image en couleur avec Numpy

Avec **np.zeros()**

```
hauteur = 100
largeur = 300
# mode RGB
canal = 3
new_array_RGB = np.zeros([hauteur, largeur, canal], dtype = np.uint8)
plt.imshow( new_array_RGB )
plt.show()
```

Ou avec **np.ones()**

```
import numpy as np
import matplotlib.pyplot as plt
# image A : dessin géométrique
dessin=np.ones((300,200,3),dtype=np.uint8)
plt.imshow( new_array_RGB )
plt.show()
```

1.Que fait le programme suivant ? (vous devrez afficher le résultat sur le plan)

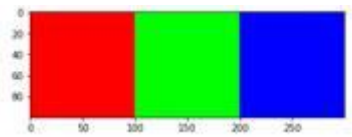
```
import numpy as np
import matplotlib.pyplot as plt
from skimage import io
# image A : dessin géométrique
dessin=255*np.ones((300,200,3),dtype=np.uint8)
```

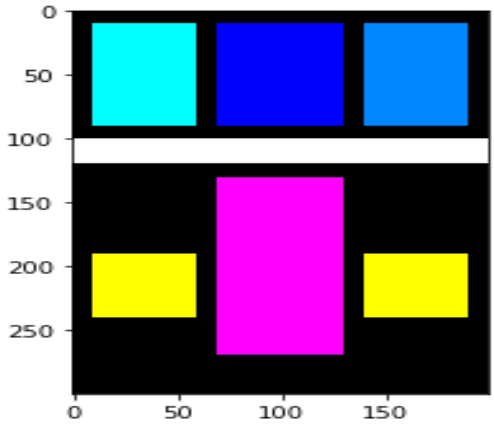
Si on ajoute les deux instructions suivants :

```
dessin[10:90,10:60,:]=(255,0,0)
dessin[10:90,140:190,:]=(255,120,0)
```

2.Quel est le résultat ?

3.Ecrire un programme qui permet de créer les images suivantes en utilisant la bibliothèque Numpy





Jaune=(255,255,0)

Rose=(255,105,180)

Bleu1(0,255,255)

Bleu2(30,144,255)

<u>Exercice 2 :</u> créer une image en nuance de gris

```
hauteur = 100
largeur = 300
# mode L
new_array_grey = np.zeros([hauteur, largeur], dtype=np.uint8)
# Noir
new_array_grey[:,:100] = 0
# Gris
new_array_grey[:,100:200] = 150
# Blanc
new_array_grey[:,200:] = 255
plt.imshow(new_array_grey, cmap = 'Greys_r')
plt.show()
```

<u>Enregistrer un vecteur NumPy sous forme d'image :</u>

Vous pouvez facilement faire le chemin inverse et créer une image PIL à partir d'un vecteur numpy en utilisant **Image.fromarray()** (Si le type de données du vecteur numpy n'est pas un entier une erreur se produira, il est donc nécessaire de le convertir en utilisant np.uint8()) :

```
PIL_image = Image.fromarray( np.uint8( image_array ) )
PIL_image

PIL_image.save("/path/nom.png")
```